

Make your first playable scene

A plain-language map of the editor, the systems behind it, and practical things you can build today.

Start here

Open the refreshed packaged editor at **Dist/ByteEngine/ByteEngine.Editor.exe**. Create a project in the Project Browser. The 3D Starter is the shortest route to a playable scene.

What this guide covers

1. Find your way around the project, scene, Assets, Hierarchy, Inspector and Game View.
2. Create a visible player from a model and use a TPS/FPS/top-down/isometric preset.
3. Add gameplay with reusable Blueprints and Event Sheets.
4. Make a health HUD, material, sound cue and socket attachment.

How the pieces fit

The mental model

Asset (model, material, animation, audio) > **Blueprint** (reusable object) > **Scene instance** (placed object) > **Components** (movement, health, camera, UI) > **Event Module** (conditions and actions) > **Play** (temporary runtime copy).

This guide is based on the current local code, including unstaged UI work. The repository README is older and describes several now-implemented features as future work. Use the editor you have just tested as the reference.

1 / Orient yourself

Open a project

1. Launch the packaged editor. In the Project Browser, create a **3D Starter** project for a ready scene, or choose **Clean Project** for an empty start.
2. Open the project's scene from **Assets/Scenes**. Keep the project **Assets** folder for imported source files and engine assets.

Know the five work areas

Area	What you do there
Assets	Import models/audio/fonts; create Blueprint, Event Module, Animation Profile and Material assets.
Hierarchy	Select scene objects and see parent-child relationships.
Inspector	Edit selected object transforms, components and asset settings.
Scene View	Place, select and frame objects while editing.
Game View / Play	See the active camera and test runtime behavior.

The edit/play boundary

Important

Save the scene and its assets before Play. Play uses a runtime copy: movement, damage and UI changes during Play are for testing; Stop does not turn them into authored scene changes.

First editor moves

Drag a model from Assets into Scene View to place it. Select an object and press **F** to frame it. Use the Inspector for exact transforms. Save the scene after meaningful edits.

2 / Build a playable character

Fast path: start from a visible model

1. Import an **FBX**, **GLB** or **gITF** character into Assets by dragging it from Explorer. Select it and check that the asset has render meshes; an animation-only FBX is not a visible player model.
2. Open a scene in **Edit** mode. Right-click the model asset and choose **Create Player Blueprint > Third Person Shooter**.
3. The editor creates a Blueprint and a scene instance, finds a clear spawn, fits the model toward character height, and frames the result. Save the scene and Blueprint.
4. Press **Play**. Click Game View to capture input. Test **WASD** move, mouse look, **Space** jump, **Left Shift** sprint and left-click fire. Press **Esc** to release captured input.

Other camera/movement presets

The same model context menu offers **First Person Shooter**, **Top-Down Twin-Stick** and **Isometric**. Or right-click empty Assets space and create a **Byte Blueprint**; its Type picker offers those presets with an empty Model child that you must fill.

What the preset assembles

A character root with movement/collision, Player Controller, Camera Boom/Camera, Health, Projectile Launcher and Player Shooter. The render model belongs on its **Model** child; the root is the movement pivot.

If the player looks tiny or sideways

First inspect the source model/import scale and the Blueprint Model child. Use the Blueprint's model-fit/orientation tools rather than compensating by scaling the character root. Check Scene View and Game View before adding animation.

3 / Blueprints, components and animation

What a Blueprint is

A **.byteblueprint** is a reusable authored object. Dragging it into a scene creates an instance. The Blueprint owns the default hierarchy and components; the scene instance can have overrides. Open the Blueprint to edit its model child, components and preview.

A useful ownership rule

Keep these jobs separate

Root: position, character movement, collision and gameplay identity.

Model child: visible imported mesh, visual fit and orientation.

Camera/boom: viewpoint and camera offset.

Health / shooter: damage state and firing.

Event Module: event-driven game rules.

Add animation without changing movement

1. In Assets, create an **Animation Profile** and open it. Choose the reference character model on the Rig tab.
2. Assign clips to Locomotion and, as needed, Actions or Blend Spaces. Save the profile.
3. Select the player root or its Animation Controller in the Blueprint/Inspector. Assign the Animation Profile asset. Save and test Play.

Troubleshooting

If movement works but the model does not animate, verify that the Animation Controller points at the intended profile and that the profile's clips belong to the intended model/skeleton. Test a source clip preview before introducing cross-rig animation.

Humanoid mapping and imported clip preview exist, but cross-rig retargeting has been troublesome in this project. Prefer clips authored for the chosen model; treat any remaining retarget workflow as advanced/experimental.

4 / Event Sheets: make objects react

The smallest useful event

An **Event Module** is a reusable **.byteevents** asset. Each event combines conditions (when?) with actions (what happens?). A module only runs when it is attached to the correct object or Blueprint.

1. Right-click empty space in Assets > **Create > Event Module**. Give it a clear name such as **TargetLogic**.
2. Double-click it to open the Event Sheet. Add an event, then choose named condition and action nodes from their categories.
3. Save the Event Module. Select the target object in Hierarchy, right-click the Event Module asset and choose **Attach to Selected GameObject**. Save the scene.

Example A: input triggers a sound

Add an **Input > Key Pressed** condition (choose a spare key), then an **Audio > Play Audio Clip** action (pick an imported clip and target/source as appropriate). Attach the module to the object that should own this logic.

Example B: reusable ray and hit result

Under **Physics / Raycasts**, choose **Cast Ray To Cursor** for cursor aiming or **Cast Ray In Direction** for a muzzle/forward ray. Follow it with **Last Raycast Hit** or **Last Raycast Missed**; **Save Raycast Result** stores hit details for later logic. **Combat / Damage > Damage Last Raycast Hit** needs a Health component on the hit object.

Avoid double firing

The playable preset already has **Player Shooter**. Adding a second Event Sheet fire action to the same input can fire twice. Disable or remove one firing path before testing your custom ray weapon.

5 / Make a health HUD

Build the Canvas

1. With a scene open, use **GameObject > UI > Text**. A Canvas is created automatically. Rename the text object **HealthText**.
2. Use **GameObject > UI > Progress Bar**. Rename it **HealthBar**. Set anchor, offsets, size and colors in the Inspector.
3. Select the Canvas if you need another reference resolution or scale mode. Text and widgets use Canvas layout; choose a suitable anchor so the HUD stays in place as the Game View size changes.

Connect it to Health

4. Create or open an Event Module that runs with your player. Add **System > Always**.
5. Add **UI > Set Text From Health**: UI Object = **HealthText**, Health Object = the player, Prefix = **HP**.
6. Add **UI > Set Bar From Health**: UI Object = **HealthBar**, Health Object = the same player. Attach the module and save.
7. Press Play and damage the player/target. The text and bar should follow current/max health.

Add a button

Create **GameObject > UI > Button**. In the widget component set its Label. An Event Sheet **UI > On Button Clicked** condition can run **Set Text**, **Show UI**, **Hide UI** or another action. Buttons also support keyboard focus and gamepad navigation.

Input rule

Hovering/clicking a UI button blocks that primary mouse click from gameplay, so pressing a menu button should not also fire the weapon.

6 / Assets that make a scene feel finished

Material in two minutes

1. Right-click empty Assets space > **Create > Material**. Open the new material.
2. Set Base Color, and optionally Normal, Metallic and Roughness values/textures. Save.
3. Assign it in the model/renderer material slot. If you want a variation, right-click the material and choose **Create Material Instance**.

Fonts and UI images

In Assets, choose **Add Free CC0 Fonts** to install the bundled set into the project, or **Import Font Files...** for custom TTF/OTF/BMFont files. Pick a Font on a Text or Button component. An Image widget picks a texture asset.

Sockets and attachments

Sockets belong to the skeletal model, not a separate attachment component. In an Animation Profile's **Sockets** tab, author a socket on the desired bone and align a preview asset. A Blueprint child can choose **Parent Socket**. Event Sheet **Attachment** actions include **Attach Object To Socket** and **Detach Object**.

Try a small example

Add **Weapon_R** on a right-hand bone. Preview a weapon to align it. Attach a weapon child to **Weapon_R**, animate the character, then test an Event Sheet switch to a back socket and Detach with Keep World.

Audio

Import an audio file into Assets, then use an Audio Source component or **Audio > Play Audio Clip** in an Event Module. A simple first use is a button click, pickup or impact cue.

7 / Diagnose problems quickly

Symptom	First check
Invisible/tiny player	Does the source model have render meshes? Check import scale, Model child, fit/orientation and Scene View framing.
Movement but no animation	Check Animation Controller profile assignment and the profile's actual clips/reference model.
Event does nothing	Save the .byteevents asset; attach it to the selected GameObject or Blueprint; check the node's target picker.
HUD not updating	Check Canvas visibility, Text/Bar target, Health Object, and whether the module is attached.
Gun fires twice	Check whether Player Shooter and Event Sheet both respond to Fire.
Ray never hits	Check ray source/muzzle, direction/cursor mode, collision layer, range and target collider.
UI button also fires	Verify you are running the latest packaged editor and the pointer is over the Button widget.

A safe learning order

1. Make one visible player. **2.** Add one collider/target with Health. **3.** Test one Event Module. **4.** Add a health HUD. **5.** Add animation, materials and sounds. **6.** Only then add socket weapons or custom ray logic.

What this guide does not promise

The current UI is a usable foundation, not a full UI toolkit: scrollable lists, text input, localization and UI animation are not implemented here. The engine remains under active development; exact panel placement may vary with your saved editor layout.

Source check

Prepared from the current ByteEngine editor/runtime code and your successful UI manual test on 27 Sep 2026. The README describes an older milestone; where it conflicts, this guide follows current code.